Real-Time Hand Gesture Recognition and Finger Counting System

Project Category: Computer Vision
Programming Language: Python
Technologies: OpenCV, MediaPipe, NumPy
Project Type: Real-Time Computer Vision Application

1. Introduction

Hand gesture recognition is an important application of Computer Vision that allows computers to understand and respond to human hand movements. It provides a natural way for people to interact with computers and electronic devices without requiring physical input devices such as keyboards or mice.

This project focuses on developing a **Real-Time Hand Gesture Recognition and Finger Counting System** using Python. The system uses a webcam to capture live video, detect human hands, identify hand landmarks, and calculate the number of raised fingers.

The project was developed using OpenCV for image processing and MediaPipe for hand detection and landmark tracking. By analysing the positions and angles of hand landmarks, the system determines which fingers are raised and displays the total finger count on the screen.

The main objective of this project is to gain practical experience in Computer Vision, real-time image processing, hand tracking, and gesture recognition while exploring their potential applications in intelligent systems.

2. Project Objectives

The main objectives of this project are:

- To develop a real-time hand detection and finger counting application.
- To understand the practical implementation of Computer Vision using Python.
- To use MediaPipe for detecting and tracking hand landmarks.
- To analyse hand movements and identify raised fingers.
- To improve recognition stability using angle-based calculations and result smoothing.
- To explore the applications of gesture-based human-computer interaction.

3. Technologies and Tools Used

The following technologies and libraries were used to develop the system.

Technology	Purpose
Python	Main programming language

OpenCV	Capturing webcam video, image processing, and displaying results
MediaPipe	Hand detection and 21-point hand landmark tracking
NumPy	Mathematical calculations and coordinate processing

3.1 Python

Python was used as the main programming language because of its simple syntax and wide range of libraries for Artificial Intelligence and Computer Vision applications.

3.2 OpenCV

OpenCV (Open Source Computer Vision Library) was used to access the webcam, capture video frames, process images, draw hand landmarks, and display the finger counting results.

3.3 MediaPipe

MediaPipe was used to detect hands and track their landmarks in real time. It identifies 21 landmarks on each detected hand, including the fingertips and finger joints. These landmarks provide the information required to analyse hand positions and movements.

3.4 NumPy

NumPy was used to perform mathematical calculations, particularly when calculating angles between hand landmarks to determine whether fingers are raised or folded.

4. Working Principle

The system follows a sequence of operations to detect hands and calculate the number of raised fingers.

Step 1 – Webcam Initialization

The webcam is initialized using OpenCV. Live video frames are continuously captured and processed by the application.

Step 2 – Image Processing

Each captured frame is converted from the BGR colour format to RGB because MediaPipe requires RGB images for hand landmark detection.

Step 3 – Hand Detection and Landmark Tracking

MediaPipe detects the hands in the video frame and identifies 21 landmarks for each detected hand. These landmarks represent important points such as the wrist, finger joints, and fingertips.

Step 4 – Finger Detection

The system analyses the positions and angles of the detected landmarks to determine whether each finger is raised or folded.

For the four fingers (index, middle, ring, and little finger), the angles formed by the finger joints are calculated. For the thumb, its angle and position are analysed using the thumb landmarks and detected hand orientation.

Step 5 – Finger Counting

The system calculates the number of raised fingers for each detected hand. If two hands are visible, their individual counts are added to calculate the total number of raised fingers.

Step 6 – Result Stabilization and Display

A short history of recent finger counts is maintained. Majority voting is used to reduce sudden changes in the displayed result. Finally, OpenCV displays the hand landmarks and total finger count on the live video.

5. Applications

Hand gesture recognition has several potential applications in modern technology.

1. **Human-Computer Interaction (HCI):** Allows users to interact with computers through hand gestures instead of traditional input devices.
2. **Touchless Control Systems:** Provides contact-free control of electronic devices, which can be useful in laboratories, hospitals, and industrial environments.
3. **Smart Home Automation:** Can be extended to control household appliances such as lights, fans, and other electrical devices using hand gestures.
4. **Assistive Technology:** Gesture-based interfaces can provide alternative interaction methods for people who experience difficulties using conventional input devices.
5. **Gaming and Virtual Reality:** Hand tracking can be used to create more natural and interactive gaming and virtual reality experiences.
6. **Sign Language Recognition:** Hand landmark detection can serve as a foundation for more advanced systems that recognize and interpret sign language gestures.
7. **Robotics:** Hand gestures can be used as commands to control robotic systems and support human-robot interaction.

6. Challenges and Improvements

During the development of the project, achieving accurate finger counting was an important challenge. Simple position-based detection methods can sometimes produce incorrect results depending on hand orientation, finger bending, and camera position.

To improve the performance, angle-based calculations were introduced to analyse finger positions more effectively. The system also uses recent detection results to reduce sudden changes in the displayed finger count.

However, the accuracy of the system can still be affected by poor lighting conditions, hand orientation, partial hand visibility, and overlapping fingers.

Possible future improvements include:

- Developing more advanced hand gesture recognition.
- Adding custom gestures to perform specific computer operations.
- Integrating the system with ESP32 or other embedded platforms.
- Using gestures to control smart home appliances.
- Developing a sign language recognition system.
- Improving recognition performance under different lighting and hand orientations.

7. Conclusion

The Real-Time Hand Gesture Recognition and Finger Counting System was successfully developed using Python, OpenCV, MediaPipe, and NumPy.

The project demonstrates how Computer Vision techniques can be used to detect human hands, track hand landmarks, analyse finger movements, and recognize simple gestures in real time.

Through this project, practical knowledge was gained in Python programming, image processing, hand landmark detection, mathematical calculations, and real-time computer vision application development.

Furthermore, the project provides a foundation for developing more advanced intelligent systems involving gesture-based control, human-computer interaction, robotics, and smart automation.